ΑΝΑΦΟΡΑ ΕΡΓΑΣΙΑΣ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ HW 1

Ονοματεπώνυμο Φοιτητή: Αλέξανδρος Γαλαζούλας

Α.Ε.Μ. : 10629

Η εργασία πραγματοποιήθηκε με τη χρήση των προγραμμάτων Quartus και ModelSim για τη δημιουργία αρχείων σε Verilog και την εξέταση της ορθής τους λειτουργίας μέσω προσομοιώσεων αντίστοιχα.

Άσκηση 1

Η σχεδίαση αφορά την υλοποίηση μιας **Arithmetic Logic Unit (ALU)** 32-bit. Αυτή η ALU είναι υπεύθυνη για την εκτέλεση αριθμητικών και λογικών πράξεων, σύμφωνα με την τιμή του σήματος ελέγχου alu_op. Η υλοποίηση της ALU χρησιμοποιεί έναν πολυπλέκτη για να επιλέγει τη λειτουργία που θα εκτελεστεί, ενώ βασίζεται σε παραμέτρους για τον καθορισμό των διαθέσιμων λειτουργιών.

Η ALU υποστηρίζει τις ακόλουθες λειτουργίες, όπως ορίζονται στην εκφώνηση και στους πίνακες:

alu_op	**Λειτουργία**	**Περιγραφή**
0000	Λογική AND	result = op1 & op2
0001	Λογική OR	result = op1 | op2
0010	Πρόσθεση	result = op1 + op2
0110	Αφαίρεση	result = op1 - op2
0100	Μικρότερο από	result = (op1 < op2) ? 1 : 0
1000	Λογική ολίσθηση δεξιά	result = op1 >> op2[4:0]
1001	Λογική ολίσθηση αριστερά	result = op1 << op2[4:0]
1010	Αριθμητική ολίσθηση δεξιά	result = $unsigned($signed(op1) >>> op2[4:0])
0101	Λογική XOR	result = op1 ^ op2

Επιπλέον, η ALU υπολογίζει ένα zero flag που τίθεται σε 1 αν το αποτέλεσμα της λειτουργίας είναι 0.

Ξεκινάω το αρχείο μου ορίζοντας τους 32 bit τελεστές εισόδου op1 και op2, μαζί με το 4 bit σήμα ελέγχου που καθορίζει τη λειτουργία. Έπειτα, ορίζω τις εξόδους μου, result (32 bit-είναι το αποτέλεσμα της λειτουργίας) και το zero (1 bit-δείχνει αν το result είναι 0). Στη συνέχεια, παραμετροποιώ τις λειτουργίες για περισσότερη ευκολία στην ανάγνωση και τέλος θέτω το result στην τιμή που πρέπει να πάρει, ανάλογα με την τιμή του alu_op. Προφανώς, λαμβάνω υπόψιν μου ότι η σύγκριση υλοποιείται με βάση προσημασμένους αριθμούς (για το ALUOP_SLT) και για την αριθμητική ολίσθηση δεξιά (ALUOP_SRA) η είσοδος op1 μετατρέπεται προσωρινά σε προσημασμένο αριθμό και το αποτέλεσμα επιστρέφεται ως μη προσημασμένο. Ως default τιμή έχω τη μηδενική, δηλαδή στην περίπτωση που το alu_op δεν παίρνει καμία από τις προαναφερθέντες τιμές.

Το αρχείο alu.v που ικανοποιεί τα παραπάνω βρίσκεται στο zip αρχείο.

Άσκηση 2

Αξιοποιώντας τώρα την ALU που δημιούργησα στην προηγούμενη άσκηση, πρέπει να φτιάξω ένα κύκλωμα αριθμομηχανής, που θα εκτελεί αριθμητικές και λογικές πράξεις, και θα διατηρεί μια τρέχουσα τιμή της σε ένα συσσωρευτή 16-bit καταχωρητή (accumulator), ενώ παράλληλα θα επιτρέπει στο χρήστη να ενημερώνει την τιμή υλοποιώντας οποιαδήποτε από τις λειτουργίες που παρέχει η ALU. Το κύκλωμα πρέπει να πληροί τις εξής απαιτήσεις:

1. Ο accumulator να μηδενίζεται σύγχρονα με το btnu και να ενημερώνεται σύγχρονα με το btnd.
2. Να συνδέονται οι είσοδοι στην ALU κατάλληλα μέσω επέκτασης προσήμου.
3. Να καθορίζεται η λειτουργία της ALU μέσω του alu_op, που παράγεται από το calc_enc.v.

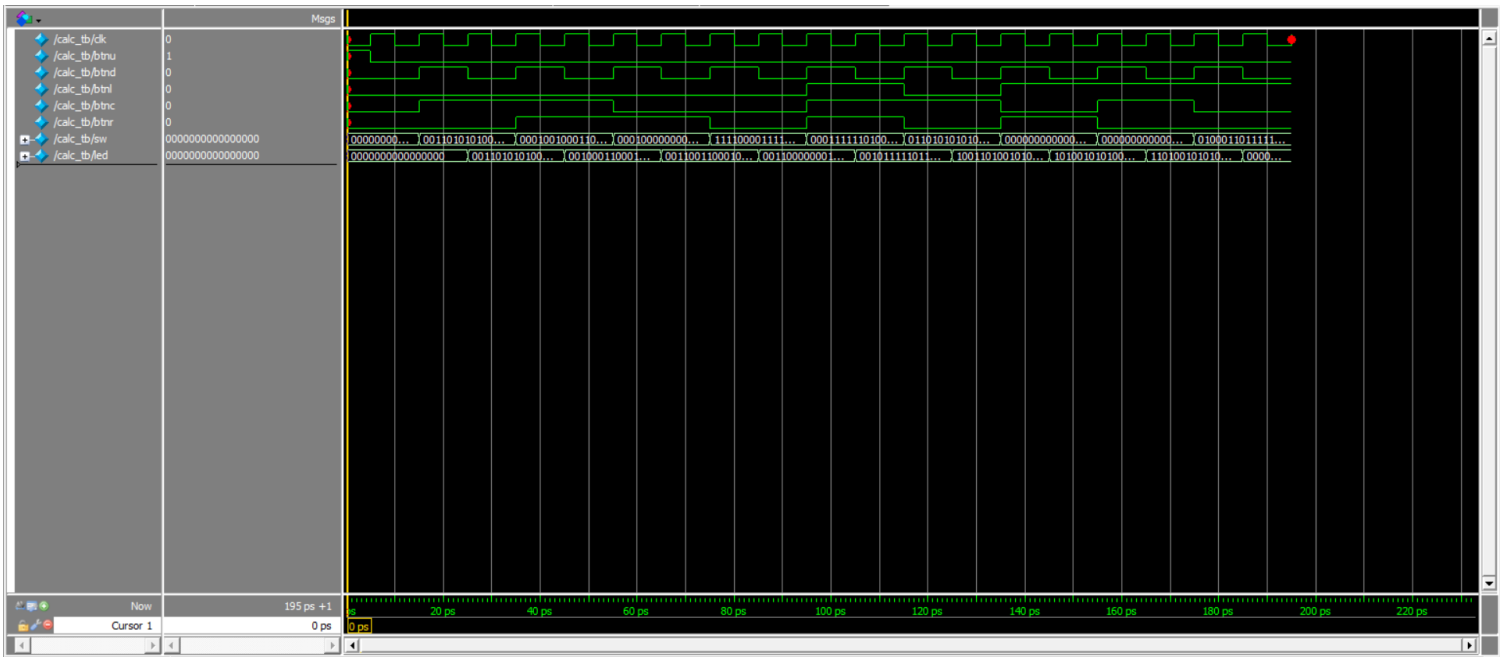
Υλοποιήθηκαν 3 αρχεία για αυτήν την άσκηση. Αρχικά, το **calc.v (Η κύρια λογική του κυκλώματος)** το οποίο περιλαμβάνει τον καταχωρητή (accumulator), τη σύνδεση με την ALU και τη λογική για τη διαχείριση των εισόδων και των εξόδων.

Τα βήματα υλοποίησης που ακολούθησα είναι τα παρακάτω:

Αρχικά δημιούργησα τον 16-bit καταχωρητή που συνδέεται με το ρολόι (clk), μηδενίζεται σύγχρονα με το btnu, ενημερώνεται μόνο όταν το btnd είναι ενεργό και η είσοδός του λαμβάνει τα 16 χαμηλότερα bits της εξόδου (result) της ALU. Ο καταχωρητής συνδέεται με την ALU δημιουργώντας δύο σήματα 32-bit με επέκταση προσήμου. Το πρώτο επεκτείνει τον καταχωρητή ({{16{accumulator[15]}}, accumulator}) και συνδέεται στο op1 της ALU και το δεύτερο επεκτείνει τα bits των διακοπτών (sw) και συνδέεται στο op2 της ALU. Το αποτέλεσμα της ALU (result) συνδέεται στα 16 λιγότερο σημαντικά bits του καταχωρητή. Η τιμή του συνδέεται απευθείας στις εξόδους LED. Επίσης, σημειώνεται ότι το alu_instance έχει δημιουργηθεί για τη σύνδεση της λειτουργίας της ALU με το υπόλοιπο κύκλωμα της αριθμομηχανής (ουσιαστικά συνδέει τις θύρες της ALU op1, op2, result, zero με τα τοπικά σήματα alu_op1, alu_op2, alu_result, alu_zero, αντίστοιχα και το alu_op θα προέλθει από το calc_enc αρχείο).

Στη συνέχεια, δημιουργώ το αρχείο **calc_enc.v** το οποίο υλοποιεί τη **συνδυαστική λογική που παράγει το alu_op** χρησιμοποιώντας τα κουμπιά btnl, btnc, και btnr. Η λογική βασίζεται στον καθορισμό των bits alu_op[0] έως alu_op[3] σύμφωνα με τα σχήματα 2-5, από τα οποία βγάζω τις σχέσεις που ισχύουν για αυτά. Το alu_op συνδέεται στην ALU μέσω του calc.v.

Τέλος, δημιουργώ το αρχείο **calc_tb.v,** το οποίο **είναι το testbench που επαληθεύει τη λειτουργία του κυκλώματος**. Περιλαμβάνει τη δημιουργία ρολογιού, την προσομοίωση των εισόδων, και τον έλεγχο της εξόδου. Σε αυτό το αρχείο βάζω πρώτα ένα σήμα ρολογιού με περίοδο 10ns, έπειτα επαληθεύω όλες τις λειτουργίες του κυκλώματος (όσες έχουν αναφερθεί στην άσκηση 1 με τη σειρά που επισημαίνει η άσκηση) και τέλος εμφανίζω τα αποτελέσματα, δηλαδή την αναμενόμενη τιμή, την πραγματική τιμή και αν πέρασε το τεστ η κάθε λειτουργία ή όχι. Η κυματομορφή που προκύπτει από το συγκεκριμένο testbench είναι η παρακάτω, στην οποία παρατηρούνται οι τιμές των κουμπιών, των διακοπτών και των led σε κάθε χρονική στιγμή της προσομοίωσης:



Άσκηση 3

Ο στόχος της άσκησης είναι η δημιουργία ενός αρχείου καταχωρητών (register file), το οποίο αποτελεί σημαντικό τμήμα του επεξεργαστή RISC-V. Το αρχείο καταχωρητών αποθηκεύει τιμές που χρησιμοποιούνται στις αριθμητικές και λογικές πράξεις, υποστηρίζοντας λειτουργίες ανάγνωσης και εγγραφής σε συγκεκριμένες διευθύνσεις.

Το αρχείο καταχωρητών υλοποιήθηκε με τις παρακάτω λειτουργίες:

1. Ανάγνωση δεδομένων από δύο θύρες ταυτόχρονα.
2. Εγγραφή δεδομένων σε μία θύρα.
3. Αρχικοποίηση όλων των καταχωρητών σε μηδενικά.
4. Διατήρηση του καταχωρητή 0 μηδενικού ανεξαρτήτως των εγγραφών.
5. Δυνατότητα προτεραιότητας εγγραφής όταν η διεύθυνση εγγραφής ταυτίζεται με διευθύνσεις ανάγνωσης.

Ξεκινάω την υλοποίηση του module regifile (αρχείο regifile.v στο zip αρχείο) ορίζοντας τα παρακάτω:

- **Είσοδοι:**
    - clk : Σήμα ρολογιού που συγχρονίζει τη λειτουργία εγγραφής.
    - readReg1, readReg2 (5-bit) : Διευθύνσεις καταχωρητών από τους οποίους διαβάζονται δεδομένα.
    - writeReg (5-bit): Διεύθυνση καταχωρητή στον οποίο γίνεται εγγραφή.
    - writeData (DATAWIDTH-bit): Τα δεδομένα προς εγγραφή.
    - write (1-bit): Σήμα ελέγχου που ενεργοποιεί την εγγραφή.
- **Έξοδοι:**
    - readData1, readData2 (DATAWIDTH-bit): Δεδομένα που ανακτώνται από τους καταχωρητές.
- **Παράμετρος DATAWIDTH:** Η παράμετρος DATAWIDTH επιτρέπει τον καθορισμό του πλάτους των καταχωρητών. Από προεπιλογή, ορίστηκε σε 32 bits.

Στη συνέχεια, δημιουργώ έναν πίνακα **καταχωρητών,** για την αποθήκευση και των 32, όπου ο καθένας θα έχει πλάτος DATAWIDTH bits και τους αρχικοποιώ με 0 (εντολή reg [DATAWIDTH-1:0] registers [31:0]). Η ανάγνωση δεδομένων από αυτούς υλοποιείται μέσω ενός always @(*) block. Όταν το σήμα write είναι ενεργό και η διεύθυνση εγγραφής ταιριάζει με μια διεύθυνση ανάγνωσης και τα δεδομένα εγγραφής εμφανίζονται απευθείας στις εξόδους ανάγνωσης.

Επιπροσθέτως, η εγγραφή γίνεται σε θετική ακμή του ρολογιού (posedge clk) όταν το σήμα write είναι ενεργό και η διεύθυνση εγγραφής δεν είναι 0 (τελευταίο always block). Τέλος, ο καταχωρητής 0 παραμένει πάντα μηδενικός, ανεξάρτητα από τις προσπάθειες εγγραφής.

Άσκηση 4

Στην Άσκηση 4, σχεδιάστηκε και υλοποιήθηκε το datapath ενός επεξεργαστή RISC-V, το οποίο υποστηρίζει βασικές λειτουργίες όπως αριθμητικές και λογικές πράξεις, προσπέλαση μνήμης, και διακλαδώσεις. Η υλοποίηση περιλαμβάνει τη χρήση της ALU που δημιουργήθηκε στην Άσκηση 1 και του αρχείου καταχωρητών από την Άσκηση 3.

Η υλοποίηση της άσκησης βασίστηκε στο περιεχόμενο του σχήματος 7 και της εκφώνησης και περιλαμβάνει τα εξής βασικά μέρη:

1) **Program Counter (PC):**

Το PC υλοποιήθηκε ως ένας καταχωρητής 32-bit, ο οποίος ενημερώνεται είτε με την τιμή PC + 4 (επόμενη εντολή) είτε με το αποτέλεσμα της διακλάδωσης (branch offset), ανάλογα με το σήμα ελέγχου PCSrc (αναφέρομαι στο σημείο του κώδικα με σχόλιο // Ενημέρωση PC). Επίσης, υπάρχει η παράμετρος INITIAL_PC με προεπιλεγμένη τιμή 0x00400000, η οποία θέτει την αρχική τιμή του PC κατά το reset. Τέλος, η λογική ενημέρωσης του PC βασίστηκε στο σήμα loadPC, το οποίο ελέγχει πότε το PC θα ανανεωθεί.

2) **Immediate Generation:**

Δημιουργήθηκαν σήματα άμεσων τιμών (immediate values) για τις εντολές τύπου I, S, και B. Αυτές παράγονται μέσω επέκτασης προσήμου-γίνεται με προσθήκη 20 άσσων μπροστά από τις εντολές I και S και 19 στις B λόγω του 0 που έχει στο τέλος-(σημείο του κώδικα με σχόλιο: // Immediate generation), σύμφωνα με το εγχειρίδιο εντολών RISC-V. Συγκεκριμένα, στις I-type εντολές, χρησιμοποιείται το πεδίο instr[31:20], στις S-type εντολές, τα πεδία instr[31:25] και instr[11:7] συνδυάζονται και στις B-type εντολές, το offset υπολογίζεται από τα πεδία instr[31], instr[30:25], και instr[11:8].

3) **Register File:**

Το αρχείο καταχωρητών από την Άσκηση 3 προστέθηκε και συνδέθηκε με τις κατάλληλες εισόδους/εξόδους. Έτσι, οι διευθύνσεις ανάγνωσης (readReg1, readReg2) και εγγραφής (writeReg) παράγονται από τα πεδία της εντολής, τα δεδομένα εγγραφής (writeData) προέρχονται είτε από την ALU είτε από τη μνήμη, ανάλογα με το σήμα MemToReg και νεργοποιείται η εγγραφή δεδομένων στους καταχωρητές μέσω του σήματος RegWrite.

4) **ALU:**

Η ALU της άσκησης 1 χρησιμοποιήθηκε για την εκτέλεση αριθμητικών και λογικών πράξεων. Ο πρώτος τελεστής (op1) προέρχεται από την έξοδο readData1 του αρχείου καταχωρητών, ενώ ο δεύτερος τελεστής (op2) επιλέγεται μέσω πολυπλέκτη. Συγκεκριμένα, προέρχεται είτε από την έξοδο readData2 είτε από την έξοδο των δεδομένων immGen, ανάλογα με το σήμα ALUSrc. Το αποτέλεσμα της ALU (ALUResult) χρησιμοποιείται για εγγραφή στους καταχωρητές ή ως διεύθυνση μνήμης.

5) **Memory Interface:**

Το αρχείο μνήμης δεδομένων ram.v, προστέθηκε στον ίδιο φάκελο με τα υπόλοιπα αρχεία της εργασίας. Έτσι, οι διευθύνσεις εγγραφής/ανάγνωσης (dAddress) προέρχονται πλέον από το αποτέλεσμα της ALU και τα δεδομένα εγγραφής (dWriteData) λαμβάνονται από το readData2 του αρχείου καταχωρητών. Επίσης, τα δεδομένα ανάγνωσης (dReadData) τροφοδοτούνται στους καταχωρητές μέσω του WriteBackData. Τέλος, το σήμα MemToReg επιλέγει αν το WriteBackData θα είναι το αποτέλεσμα της ALU ή τα δεδομένα από τη μνήμη.

6) **Branch Target:**

Υπολογίστηκε ο στόχος διακλάδωσης προσθέτοντας το PC με την άμεση τιμή των B-type εντολών (branch_offset).Το σήμα PCSrc ελέγχει αν το PC θα ενημερωθεί με το PC + 4 ή τη διεύθυνση διακλάδωσης.

7) **Write Back:**

Ένας πολυπλέκτης χρησιμοποιήθηκε για να επιλέγει μεταξύ του αποτελέσματος της ALU και των δεδομένων ανάγνωσης μνήμης (dReadData) για εγγραφή στο αρχείο καταχωρητών. Η έξοδός του συνδέθηκε επίσης στο σήμα WriteBackData.

Άσκηση 5

Η υλοποίηση του πολύκυκλου ελεγκτή (FSM) βασίζεται στη δομή του datapath που σχεδιάστηκε στην Άσκηση 4. Το datapath παρέχει τη βασική υποδομή για την εκτέλεση εντολών, ενώ το FSM ελέγχει τη ροή των δεδομένων και τα σήματα ελέγχου για την ολοκλήρωση κάθε εντολής σε πέντε κύκλους: Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), και WriteBack (WB). Το FSM σχεδιάστηκε από τα παρακάτω στάδια:

1. Fetch (IF):

Στο στάδιο Instruction Fetch, ο επεξεργαστής φορτώνει την επόμενη εντολή από τη μνήμη εντολών. Το PC αυξάνεται κατά 4 ή ενημερώνεται με βάση τη διακλάδωση (σχήμα 7 της εκφώνησης). Χρησιμοποιήθηκε η παράμετρος INITIAL_PC για την εκκίνηση του προγράμματος (0x00400000), όπως υλοποιήθηκε στην Άσκηση 4.

2. Decode (ID):

Στο στάδιο Decode, αποκωδικοποιείται η εντολή και δημιουργούνται τα Immediate Values για I, S, και B-type εντολές, σύμφωνα με την Άσκηση 4. Τα δεδομένα από το αρχείο καταχωρητών φορτώνονται στους πρώτους τελεστές της ALU (readData1 και readData2).

3. Execute (EX):

Στο στάδιο Execute, η ALU εκτελεί την πράξη που καθορίζεται από το opcode και τα funct3/funct7 πεδία της εντολής. Δόθηκε περεταίρω έμφαση στις διακλαδώσεις (BEQ), όπου το αποτέλεσμα της ALU (Zero) χρησιμοποιείται για να καθορίσει την ενημέρωση του PC.

4. Memory Access (MEM):

Στο στάδιο MEM, η μνήμη δεδομένων (DATA_MEMORY) χρησιμοποιείται για ανάγνωση (MemRead) ή εγγραφή (MemWrite). Οι διευθύνσεις μνήμης παράγονται από την ALU (dAddress).

5. WriteBack (WB):

Στο στάδιο WB, το αποτέλεσμα της ALU ή τα δεδομένα ανάγνωσης από τη μνήμη (dReadData) γράφονται πίσω στο αρχείο καταχωρητών. Ένας πολυπλέκτης καθορίζει την πηγή των δεδομένων, βάσει του σήματος MemToReg.

Τα σήματα ελέγχου που παράγει το FSM για να συντονίζει τις λειτουργίες του datapath είναι τα παρακάτω:

1. **PCSrc:** Ενημερώνει το PC με τη διεύθυνση διακλάδωσης ή την επόμενη εντολή.
2. **ALUSrc:** Καθορίζει αν ο δεύτερος τελεστής της ALU είναι immediate value ή δεδομένα από τον καταχωρητή.
3. **MemRead και MemWrite:** Ενεργοποιούν τις αντίστοιχες λειτουργίες της μνήμης δεδομένων.
4. **RegWrite:** Ενεργοποιεί την εγγραφή στο αρχείο καταχωρητών κατά το στάδιο WB.
5. **ALUCtrl:** Καθορίζει τη λειτουργία της ALU για αριθμητικές, λογικές, και εντολές διακλάδωσης.

Σημειώνεται ότι οι εντολές του αρχείου rom_bytes.data αποκωδικοποιήθηκαν και ενσωματώθηκαν απευθείας στο αρχείο INSTRUCTION_MEMORY.v και τα αρχεία rom.v και ram.v ονομάστηκαν ως INSTRUCTION_MEMORY.v και DATA_MEMORY.v διότι υπήρχε σύγχυση με τα ονόματα των αρχείων και δε μπορούσαν να τρέξουν στο πρόγραμμα προσομοίωσης (βρίσκονται στο φάκελο με αυτή την ονομασία).

Τέλος, παρόλη την υλοποίηση που υπάρχει στα αρχεία top_proc.v και top_proc_tb.v, ο ελεγκτής δε μπόρεσε να λειτουργήσει όπως αναμενόταν. Συγκεκριμένα, δεν υπάρχει σωστή σύνδεση μεταξύ ALU ή μνήμης και του σήματος WriteBackData και δεν αποστέλλονται έγκυρες διευθύνσεις/δεδομένα στη μνήμη, ίσως λόγω προβλήματος στον πολυπλέκτη της ALU. Επίσης, δεν ενεργοποιούνται όπως πρέπει στις εντολές LW και SW, υπάρχει εσφαλμένη εφαρμογή των πράξεων για συγκεκριμένες τιμές funct3/funct7 και το RegWrite δεν φαίνεται να ενεργοποιεί σωστά την εγγραφή στους καταχωρητές κατά τη φάση WB.